

Load Balancer vs API Gateway

How a Load Balancer and API Gateway Work Together

Pawan Natekar | Devops | AWS | K8s | Linux

Published July 23, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

Why Were They Created?	3
What Is a Load Balancer?	4
Main responsibilities	4
Example of a Load Balancer	4
What Is an API Gateway?	5
Main Responsibilities of an API Gateway	6
Example of an API Gateway	6
Load Balancer vs API Gateway	7
How They Work Together	7
When Should You Use a Load Balancer?	8
When Should You Use an API Gateway?	8
Can You Use Both?	9

Every time you order food online, book a cab, or watch a video, thousands of requests are processed in the background.

Yet everything feels smooth.

How does that happen?

Modern applications use different components to manage traffic, improve performance, and protect their services.

Two of the most important are the Load Balancer and the API Gateway.

Although beginners often confuse them, they solve different problems.

Let's understand them step by step.

Why Were They Created?

Imagine you are using Amazon during a big sale like Prime Day.

Millions of people are searching for products, adding items to their carts, and placing orders at the same time.

If every request were sent to a single server, the website would quickly become slow or even stop responding.

To prevent this, Amazon uses multiple servers to handle the traffic.

A **Load Balancer** sits in front of these servers and distributes incoming requests so that no single server becomes overloaded.

Now think about what happens after you click **Place Order**.

Your request does not go to just one service.

It may need to communicate with several services, such as:

- User Service to verify your account
- Product Service to check product details
- Inventory Service to confirm stock
- Payment Service to process your payment
- Order Service to create your order
- Notification Service to send a confirmation email

Imagine if the Amazon app had to know the address of every one of these services and communicate with each one directly.

That would make the application more complex, harder to maintain, and less secure.

Instead, the app sends the request to an **API Gateway**.

The API Gateway becomes the single entry point. It authenticates the request, applies security rules if needed, and forwards it to the correct services.

This is why both components exist.

A **Load Balancer** helps applications handle large amounts of traffic.

An **API Gateway** helps applications manage and secure communication with multiple services.

What Is a Load Balancer?

A Load Balancer distributes incoming traffic across multiple servers.

Instead of sending every request to one machine, it decides which server should handle the next request.

This improves performance, reliability, and availability.

If one server becomes unavailable, requests are automatically sent to healthy servers.

Think of it as a traffic police officer who directs vehicles toward different roads to prevent traffic jams.

Main responsibilities

- Distributes incoming traffic
- Prevents one server from becoming overloaded
- Improves application availability
- Performs health checks
- Redirects traffic when a server fails

Example of a Load Balancer

Imagine an online shopping website.

Three web servers are running the same application.

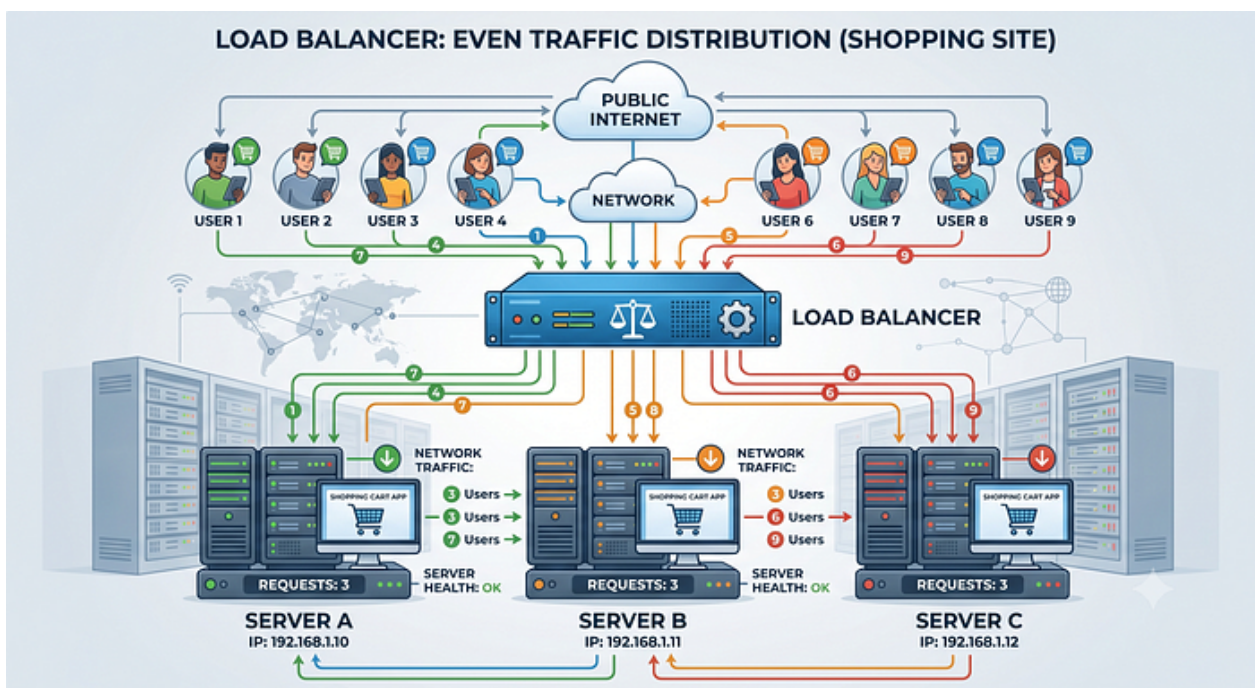
- Server A
- Server B
- Server C

When 9 users visit the website, the Load Balancer may distribute requests like this.

- User 1 → Server A
- User 2 → Server B
- User 3 → Server C
- User 4 → Server A
- User 5 → Server B
- User 6 → Server C

Instead of one server handling everything, the work is shared.

As a result, the website stays fast even during busy hours.



What Is an API Gateway?

An API Gateway is a single entry point for API requests.

Instead of clients calling every microservice directly, they send requests to the API Gateway.

The gateway decides where the request should go.

Before forwarding the request, it can perform several additional tasks.

It can verify authentication, check permissions, apply rate limits, transform requests, and collect logs.

Think of it as a receptionist in an office.

Visitors first meet the receptionist.

The receptionist checks their identity, understands where they need to go, and then sends them to the correct department.

Main Responsibilities of an API Gateway

- Routes API requests
- Handles authentication
- Checks authorization
- Applies rate limiting
- Supports API versioning
- Collects logs and metrics
- Transforms requests and responses
- Hides internal services from clients

Example of an API Gateway

Suppose you open a food delivery application.

The mobile app requests your dashboard.

Behind the scenes, the application needs information from multiple services.

- User Service
- Restaurant Service
- Order Service
- Payment Service

Instead of making four separate requests, the mobile app sends one request to the API Gateway.

The gateway communicates with all required services and returns a single response.

The client receives everything through one endpoint.

This makes the application easier to maintain and use.

Load Balancer vs API Gateway

LOAD BALANCER VS API GATEWAY		
FEATURE	LOAD BALANCER	API GATEWAY
MAIN PURPOSE	DISTRIBUTES TRAFFIC	MANAGES API REQUESTS
WORKS WITH	SERVERS	APIS AND MICROSERVICES
AUTHENTICATION	USUALLY NO	YES
RATE LIMITING	USUALLY NO	YES
API ROUTING	BASIC	ADVANCED
HEALTH CHECKS	YES	SOMETIMES
SSL TERMINATION	YES	YES
REQUEST TRANSFORMATION	NO	YES
API VERSIONING	NO	YES
BEST FOR	HIGH AVAILABILITY	API MANAGEMENT

How They Work Together

Consider an e-commerce application.

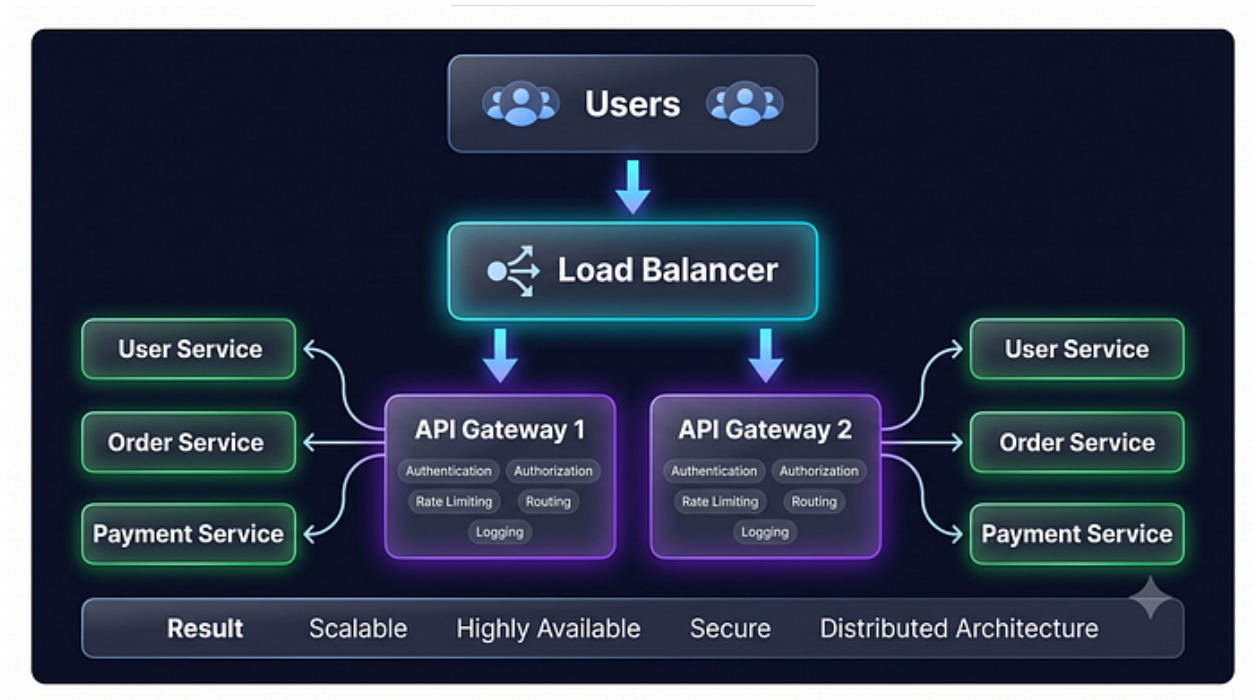
Thousands of users access the application every minute.

The traffic first reaches the Load Balancer.

The Load Balancer distributes requests across multiple API Gateway instances.

Each API Gateway then performs authentication, checks permissions, applies rate limits, and forwards requests to the correct microservice.

The request flow looks like this.



The Load Balancer ensures no gateway becomes overloaded.

The API Gateway ensures every request is secure and reaches the correct service.

When Should You Use a Load Balancer?

A Load Balancer is the right choice when you need to:

- Run multiple application servers
- Improve availability
- Prevent server overload
- Handle hardware or server failures
- Scale applications during heavy traffic

When Should You Use an API Gateway?

An API Gateway is useful when you need to:

- Build microservices
- Secure APIs
- Authenticate users
- Control API access
- Apply rate limits
- Hide internal services
- Manage multiple API versions

Can You Use Both?

Yes.

Large applications commonly use both because they solve different problems.

A Load Balancer keeps traffic evenly distributed.

An API Gateway manages and secures API communication.

Together, they create applications that are scalable, reliable, and easier to manage.